

Software Requirements Specification

(SRS)

Project: Collaborative Study Material and Quiz Platform

Subject: Software Engineering (UE24CS341A)

Version: 1.0

Prepared by:

SADANAND PES2UG24CS423

SAGAR K PES2UG24CS424

SAHANA K S PES2UG24CS425

SAHANA M S PES2UG24CS426

Team / Group No.: 13

Date: 03-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary	Approval
1.0	03-06-2026	Team 13	Initial SRS draft	Pending

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Team Lead			

Table of Contents

- 1. Introduction
 - 1.1 Purpose
 - 1.2 Scope
 - 1.3 Intended Audience
 - 1.4 Definitions, Acronyms and Abbreviations
 - 1.5 References
- 2. Overall Description
- 3. External Interface Requirements
- 4. System Features (Detailed Functional Requirements)
 - 4.1 User Authentication & Role Management
 - 4.2 Study Material Sharing
 - 4.3 Quiz Management
 - 4.4 Question Bank
 - 4.5 Performance Analysis
 - 4.6 Collaboration & Notifications
- 5. Non-Functional Requirements
 - 5.1 Security
- 6. Quality Attributes & Acceptance Tests
- 7. System Models and Diagrams (UML Use-Case Diagrams)
- 8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for the Collaborative Study Material and Quiz Platform, a web application that allows students and instructors to share study material, build and attempt quizzes, maintain a reusable question bank, and track academic performance through analytics. This document is intended to guide the design, implementation, and testing of the system for the Software Engineering mini-project (UE24CS341A).

1.2 Scope

The system is a web-based platform, built on the MERN stack (MongoDB, Express.js, React.js, Node.js) with a supporting Python service for performance analytics, that enables:

- Uploading, organizing, and sharing study material by course/subject/topic.
- Creation, scheduling, and attempting of quizzes with automatic grading of objective questions.
- Maintenance of a searchable, reusable question bank tagged by subject, topic, and difficulty.
- Generation of individual and class-level performance analytics and reports.
- Basic collaboration features such as comments and ratings on shared material.

Out of scope for this version: payment/subscription handling, plagiarism-detection engines, integration with external LMS platforms, and native mobile applications (the web application will be responsive for mobile browsers).

1.3 Intended Audience

Developers and the project team, course instructors/evaluators, QA/testers, and end users (students and instructors) who will validate that the delivered features meet the stated requirements.

1.4 Definitions, Acronyms and Abbreviations

- MERN – MongoDB, Express.js, React.js, Node.js
- MCQ – Multiple Choice Question
- JWT – JSON Web Token
- RBAC – Role-Based Access Control
- API – Application Programming Interface
- CSV – Comma-Separated Values
- CI/CD – Continuous Integration / Continuous Deployment
- UI/UX – User Interface / User Experience

1.5 References

- IEEE SRS Template (course-provided).
- Software Engineering Mini-Project Guidelines, UE24CS341A (course-provided).

2. Overall Description

2.1 Product Perspective

The platform is a new, self-contained web application. It consists of a React.js single-page front end, a Node.js/Express REST API backend, a MongoDB database, and a Python microservice (Flask/FastAPI) that computes and serves performance-analytics data over REST. All components communicate over HTTPS within the same deployed environment.

2.2 Product Functions (Summary)

- User registration, login, and role-based access (Student, Instructor, Admin).

- Study material upload, categorization, search, download, and bookmarking.
- Quiz creation, scheduling, attempting, and automatic/manual grading.
- Question bank creation, tagging, search, and bulk import.
- Individual and class-level performance analytics and exportable reports.
- Comments and ratings on study material; in-app/email notifications.
- Administrative moderation of users and content.

2.3 User Classes and Characteristics

- Student: primary consumer of material and quizzes; expects a simple, fast, mobile-friendly experience; limited technical expertise assumed.
- Instructor: creates/manages material, quizzes, and the question bank; needs efficient authoring tools and clear analytics.
- Admin: manages users, moderates content, and monitors overall system health; highest privilege level.

2.4 Operating Environment

Client: modern web browsers (Chrome, Firefox, Edge) on desktop and mobile. Server: Node.js LTS runtime hosting the Express REST API; Python 3.x runtime hosting the analytics microservice. Database: MongoDB (local instance or MongoDB Atlas). Deployment target: a cloud platform (e.g., Render, Railway, AWS, or similar) with HTTPS enabled.

2.5 Design and Implementation Constraints

- Core application must be implemented using the MERN stack; analytics module implemented in Python.
- Only open-source libraries/frameworks may be used, in line with an academic project timeline.
- Agile methodology with Git/GitHub for version control and a Jira-like tool for backlog/user-story tracking, as mandated by the course guidelines.
- CI/CD pipeline, automated tests, and static code analysis are required deliverables.
- Uploaded file size and type restrictions apply to manage storage costs (e.g., PDFs, DOCX, PPTX, images up to a configured limit).

2.6 Assumptions and Dependencies

- Users have consistent access to the internet and a supported browser.
- Cloud/file storage (e.g., AWS S3, Cloudinary, or MongoDB GridFS) is available for storing uploaded material.
- An SMTP provider (e.g., Gmail SMTP, SendGrid) is available for email notifications and password resets.
- The Python analytics service is reachable by the Node.js backend over an internal REST API.

3. External Interface Requirements

3.1 User Interfaces

A responsive React.js single-page application with role-specific dashboards (Student / Instructor / Admin). Common UI elements include a persistent navigation bar, search bar, notification icon, and consistent form validation/error messaging across the app. Screens include: Login/Register, Material Library, Material Detail, Quiz List, Quiz Attempt, Quiz Result, Question Bank Manager, Analytics Dashboard, and Admin Panel.

3.2 Hardware Interfaces

None beyond a standard computer or mobile device capable of running a modern web browser; no specialised hardware is required.

3.3 Software Interfaces

- Frontend ↔ Backend: REST API over HTTPS, JSON payloads (React.js ↔ Node.js/Express).
- Backend ↔ Database: MongoDB via Mongoose ODM.
- Backend ↔ Analytics Service: internal REST API (Node.js ↔ Python Flask/FastAPI) for computing performance statistics.
- Backend ↔ File Storage: cloud object storage or GridFS for uploaded study material.
- Backend ↔ Email Service: SMTP/third-party email API for notifications and password resets.

3.4 Communications Interfaces

- HTTPS (TLS 1.2+) for all client-server and inter-service communication.
- JWT bearer tokens for authenticated API requests.
- Optional WebSocket channel for real-time in-app notifications.

<< Note: Overall this project defines 18 Functional Requirements, 5 Non-Functional Requirements, 3 Security Objectives, and 5 Security Requirements, as detailed in Sections 4 and 5. >>

4. System Features (Detailed Functional Requirements)

Requirement IDs follow the pattern CSQP-F-####. Each requirement includes an acceptance criterion and a reference test case, to be elaborated in the Test Plan (Jackfruit Phase-3).

4.1 User Authentication & Role Management

Registration, login, role-based access control, and password recovery for Students, Instructors, and Admins.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-001	The system shall allow a new user to register with name, email, password, and role (Student or Instructor).	Functional	High	All Users	AC-001: Valid registration creates an account and redirects to login. Test: TC-Auth-01	Admin role assigned manually / seeded.
CSQP-F-002	The system shall authenticate users via email/password and issue a JWT on successful login.	Functional	High	All Users	AC-002: Valid credentials return a JWT and profile info. Test: TC-Auth-02	Token expiry configurable.
CSQP-F-003	The system shall enforce role-based access control (RBAC) so that only Instructors can create/edit quizzes and material, and only Admins can manage users.	Functional	High	Security	AC-003: Unauthorized role attempting a restricted action receives HTTP 403. Test: TC-Auth-03	Middleware on all protected routes.
CSQP-F-004	The system shall allow a user to reset a forgotten password via an emailed reset link/OTP.	Functional	Medium	Student / Instructor	AC-004: Reset link expires after 30 minutes; new password takes effect immediately. Test: TC-Auth-04	Requires SMTP integration.

4.2 Study Material Sharing

Uploading, browsing, searching, downloading, bookmarking, and managing shared study material.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-010	The system shall allow an Instructor to upload study material (PDF/DOCX/PPT or	Functional	High	Instructor	AC-010: Uploaded file appears in the material library with correct tags. Test: TC-Mat-01	File size/type validation required.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
	external link) tagged with course, subject, and topic.					
CSQP-F-011	The system shall allow Students to browse and search material by course, subject, topic, or keyword.	Functional	High	Student	AC-011: Search returns results matching the query within 2 seconds for a seeded dataset. Test: TC-Mat-02	Depends on indexing strategy.
CSQP-F-012	The system shall allow Students to view/download material and shall record a download count per item.	Functional	High	Student	AC-012: Download completes successfully and the counter increments by 1. Test: TC-Mat-03	-
CSQP-F-013	The system shall allow Students to bookmark material for quick access from a personal "Saved Material" list.	Functional	Medium	Student	AC-013: Bookmarked item appears in the Saved Material list and can be removed. Test: TC-Mat-04	-
CSQP-F-014	The system shall allow the uploading Instructor or an Admin to edit metadata or delete a material item, maintaining a version/audit note.	Functional	Medium	Instructor / Admin	AC-014: Edited/deleted items reflect immediately for all users; an audit log entry is created. Test: TC-Mat-05	Soft delete recommended.

4.3 Quiz Management

Creation, scheduling, attempting, grading, and result review of quizzes.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-020	The system shall allow an Instructor to create a quiz consisting of MCQ, True/False, and short-answer questions, with a configurable duration and passing score.	Functional	High	Instructor	AC-020: Created quiz is saved with all configured questions and settings. Test: TC-Quiz-01	Uses Question Bank items or new questions.
CSQP-F-021	The system shall allow an Instructor to schedule a quiz with a start and end date/time, visible to assigned students only within that window.	Functional	High	Instructor	AC-021: Quiz is inaccessible before the start time and after the end time. Test: TC-Quiz-02	-
CSQP-F-022	The system shall allow a Student to attempt an active quiz within the allotted time, auto-submitting when the timer expires.	Functional	High	Student	AC-022: Quiz auto-submits at time-out and captures all answered questions. Test: TC-Quiz-03	Client-side timer with server-side validation.
CSQP-F-023	The system shall automatically grade objective questions (MCQ/True-False) immediately upon submission.	Functional	High	Student / Instructor	AC-023: Auto-graded score matches the answer key for a test quiz. Test: TC-Quiz-04	-
CSQP-F-024	The system shall allow an Instructor to manually grade subjective (short-answer) questions and shall update the final score accordingly.	Functional	Medium	Instructor	AC-024: Manually entered marks are added to the auto-graded score and reflected in the student's result. Test: TC-Quiz-05	-

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-025	The system shall allow a Student to view their quiz result and, if enabled by the Instructor, the correct answers after submission.	Functional	Medium	Student	AC-025: Result screen shows score and (if enabled) an answer review. Test: TC-Quiz-06	Instructor-configurable visibility flag.
CSQP-F-026	The system shall allow an Instructor to view all student attempts for a quiz and export the results as CSV.	Functional	Medium	Instructor	AC-026: Exported CSV contains one row per student attempt with score and timestamp. Test: TC-Quiz-07	-

4.4 Question Bank

Centralised, reusable, and searchable repository of questions for quiz authoring.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-030	The system shall allow an Instructor to add, edit, or delete a question with metadata: subject, topic, difficulty, and type.	Functional	High	Instructor	AC-030: New/edited question is saved and retrievable by its metadata. Test: TC-QB-01	-
CSQP-F-031	The system shall allow an Instructor to search and filter the question bank by tag, subject, topic, or difficulty for reuse in a quiz.	Functional	High	Instructor	AC-031: Filtered results match all applied filter criteria. Test: TC-QB-02	-
CSQP-F-032	The system shall allow bulk import of questions via a CSV/Excel template.	Functional	Medium	Instructor	AC-032: Valid file imports all rows as questions; invalid rows are reported with a reason. Test: TC-QB-03	Template published in Appendix B.
CSQP-F-033	The system shall flag likely duplicate questions (based on text similarity) when a new question is added.	Functional	Low	Instructor	AC-033: Adding a near-identical question surfaces a duplicate warning before saving. Test: TC-QB-04	Basic string-similarity check; optional enhancement.

4.5 Performance Analysis

Individual and class-level analytics dashboards and exportable reports, powered by the Python analytics service.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-040	The system shall generate an individual performance report for a Student showing score trends and topic-wise strengths/weaknesses.	Functional	High	Student	AC-040: Report reflects all completed quiz attempts for the student, grouped by topic. Test: TC-Perf-01	Served by the Python analytics microservice.
CSQP-F-041	The system shall provide an Instructor with class-level analytics, including average score, score distribution, and per-question success rate.	Functional	High	Instructor	AC-041: Analytics values match a manually computed sample dataset within rounding tolerance. Test: TC-Perf-02	-
CSQP-F-042	The system shall present analytics using charts (e.g.,	Functional	Medium	Student / Instructor	AC-042: Dashboard renders charts correctly for a seeded	Recharts/Chart.js on the React front end.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
	line/bar) on a dashboard for both Students and Instructors.				dataset without errors. Test: TC-Perf-03	
CSQP-F-043	The system shall allow performance reports to be exported as PDF or CSV.	Functional	Medium	Student / Instructor	AC-043: Exported file opens correctly and matches the on-screen report. Test: TC-Perf-04	-

4.6 Collaboration & Notifications

Comments, ratings, and notifications that support peer collaboration around shared material.

Req ID	Requirement (shall...)	Type	Priority	Source / Stakeholder	Acceptance Criteria / Test Ref	Comments / Dependencies
CSQP-F-050	The system shall allow Students and Instructors to post comments on a study material item.	Functional	Medium	Student / Instructor	AC-050: Posted comment appears immediately under the material item for all viewers. Test: TC-Collab-01	Basic profanity filter recommended.
CSQP-F-051	The system shall send an in-app and/or email notification to relevant Students when new material or a new quiz is published.	Functional	Medium	Student	AC-051: Notification is generated within 1 minute of publishing. Test: TC-Collab-02	Requires SMTP; WebSocket optional for in-app.
CSQP-F-052	The system shall allow Students to rate study material on a 1-5 star scale, and display the average rating.	Functional	Low	Student	AC-052: Average rating updates immediately after a new rating is submitted. Test: TC-Collab-03	-

5. Non-Functional Requirements

NFRs below are measurable and tied to the project's test plan. IDs follow CSQP-NF-###.

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
CSQP-NF-001	90% of page loads and API responses shall complete within 2 seconds under normal load (up to 100 concurrent users).	Performance	High	Load test (e.g., JMeter/k6) shows 90th-percentile response $\leq$ 2s. Test: TC-Perf-NF-01
CSQP-NF-002	The system shall support at least 100 concurrent users without service degradation during a scheduled class quiz window.	Scalability	High	Load test with 100 simulated concurrent quiz attempts completes without errors. Test: TC-Scale-01
CSQP-NF-003	The system shall be available 99% of the time during active semester hours, excluding scheduled maintenance.	Reliability / Availability	Medium	Uptime monitoring report over a 2-week observation period. Test: Ops log review
CSQP-NF-004	The web UI shall be responsive and usable on both desktop and mobile browsers, and shall follow basic WCAG 2.1 AA guidance (colour contrast, keyboard navigation).	Usability / Accessibility	Medium	Manual accessibility checklist and responsive-layout check across 3 screen sizes. Test: TC-UX-01
CSQP-NF-005	Codebase shall maintain a minimum of 70% unit test coverage and pass CI checks (build, lint, test) before merge.	Maintainability	High	CI pipeline (e.g., GitHub Actions/Jenkins) coverage report $\geq$ 70%. Test: CI report

5.1 Security

5.1.1 Security Objectives

- Protect student and instructor personal data and credentials from unauthorized access or disclosure.
- Ensure the integrity of quiz content, submissions, and results so they cannot be tampered with by unauthorized parties.
- Restrict access to Instructor- and Admin-only functions to properly authenticated and authorized users (RBAC enforcement).

5.1.2 Security Requirements

Req ID	Requirement (shall...)	Type	Priority	Acceptance Criteria / Test Ref
CSQP-SR-001	TLS 1.2+ shall be mandatory for all client-server and inter-service network connections.	Security	High	AC: All endpoints reject plain HTTP in production. Test: TC-Sec-01
CSQP-SR-002	User passwords shall be hashed (e.g., bcrypt) and never stored or logged in plaintext.	Security	High	AC: Database inspection shows only hashed values. Test: TC-Sec-02
CSQP-SR-003	The system shall issue short-lived JWT access tokens with a refresh-token mechanism for session renewal.	Security	High	AC: Expired access tokens are rejected; refresh flow issues a new valid token. Test: TC-Sec-03
CSQP-SR-004	Every protected API route shall validate the caller's role via RBAC middleware before processing the request.	Security	High	AC: Requests from an unauthorized role return HTTP 403 for all protected routes. Test: TC-Sec-04
CSQP-SR-005	All user input shall be validated and sanitized on the server to mitigate injection and XSS attacks, and authentication endpoints shall be rate-limited.	Security	High	AC: Automated security scan (e.g., OWASP ZAP) reports no high-severity injection/XSS findings; brute-force login attempts are throttled. Test: TC-Sec-05

6. Quality Attributes & Acceptance Tests

6.1 Exit Criteria for Acceptance

- All High-priority functional requirements are implemented and verified.
- No critical or High-priority NFR/security requirement is failing.
- The Requirements Traceability Matrix (Section 8) shows all associated test cases passed.

6.2 Acceptance Test Suites

- Authentication & Access Control
- Study Material Management
- Quiz Creation, Attempt & Grading
- Question Bank
- Performance Analytics
- Security
- Usability / Accessibility

7. System Models and Diagrams

7.1 UML Use-Case Diagrams

The following use-case diagrams summarise the primary interactions of the Student and Instructor actors with the system. These are starting diagrams for the team to refine during the Design phase (Jackfruit Phase-1/2).

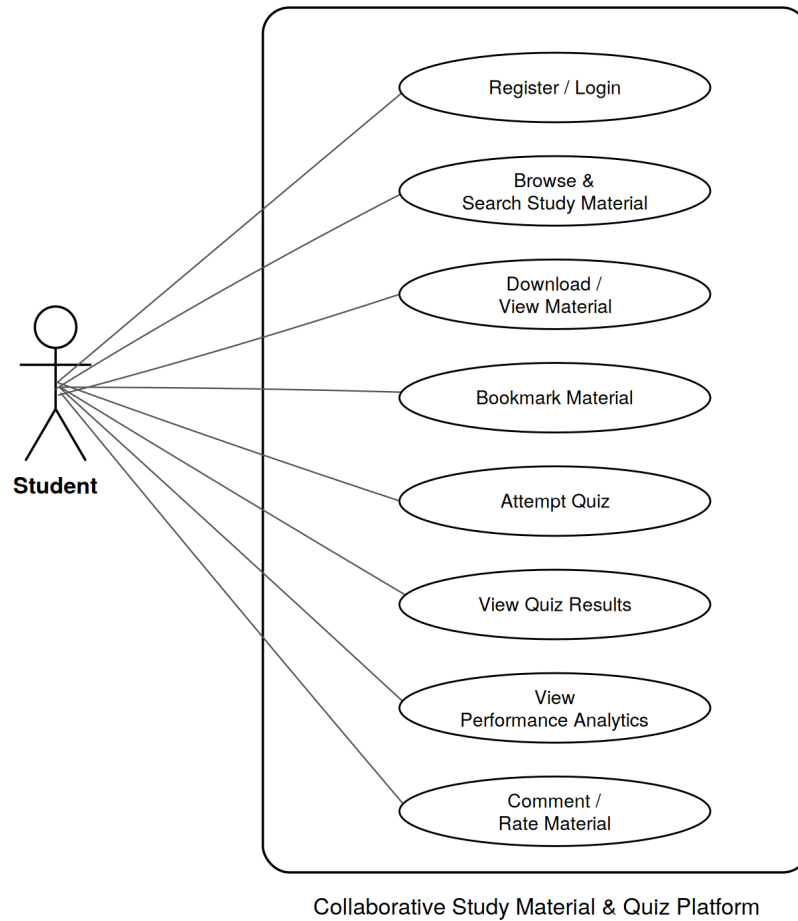


Figure 1: Use-Case Diagram – Student

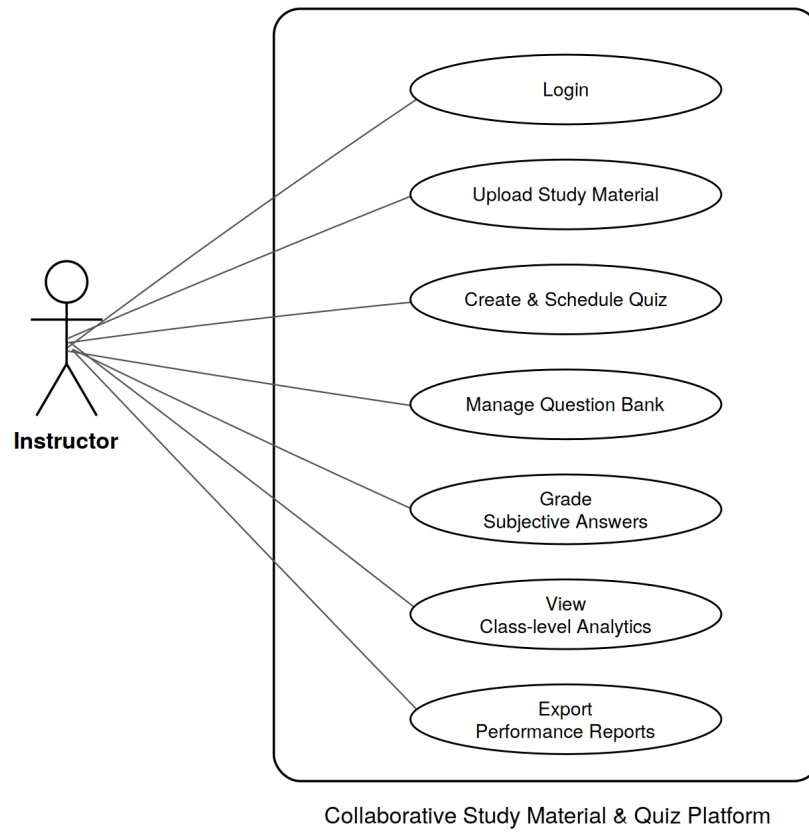


Figure 2: Use-Case Diagram – Instructor

8. Requirements Traceability Matrix (RTM)

Status legend: N = Not started, P = In progress, A = Achieved/Passed. This matrix will be updated through the Implementation and System Validation phases (Jackfruit Phase-2/3).

Req ID	Requirement (short)	Section Ref	Module	Test Case(s)	Status	Comments
CSQP-F-002	Login with JWT	4.1 / DS-Auth-01	AuthModule	TC-Auth-02	N	
CSQP-F-003	RBAC enforcement	4.1 / DS-Auth-02	AuthModule	TC-Auth-03	N	
CSQP-F-010	Upload study material	4.2 / DS-Mat-01	MaterialModule	TC-Mat-01	N	
CSQP-F-020	Create quiz	4.3 / DS-Quiz-01	QuizModule	TC-Quiz-01	N	
CSQP-F-022	Attempt quiz (timed)	4.3 / DS-Quiz-02	QuizModule	TC-Quiz-03	N	
CSQP-F-030	Manage question bank	4.4 / DS-QB-01	QuestionBankModule	TC-QB-01	N	
CSQP-F-040	Individual performance report	4.5 / DS-Perf-01	AnalyticsModule (Python)	TC-Perf-01	N	
CSQP-NF-001	Response time target	5 / DS-Perf-02	Web UI / API	TC-Perf-NF-01	N	
CSQP-SR-002	Password hashing	5.1.2 / DS-Sec-01	AuthModule	TC-Sec-02	N	